

Scratchpad MNIST

Konstantin Zeck

June 2026

1 Forward Pass

Using indices: $i \in \{1, \dots, 784\}$ with **784** being the number of pixels per image – each pixel has a value $v \in [0, 1]$, $j \in \{1, \dots, 256\}$ with **256** being the number of hidden dimensions, and finally $k \in \{1, \dots, C\}$ with **C = 10** being the number of classes.

MPI distributed DDP parallelism

A batch of (e.g. 128 images) is split among R MPI Processes such that in the end every gradient update using **SGD** has to account for this by doing a **MPI_Allreduce** with **MPI_SUM** (equivalent to $\frac{1}{R} \sum_{r=1}^R$).

Note 1.1: The forward computation is per sample s .

1.1 1st Layer

Hidden layer output

$$H_j^{(s)} = X_i^{(s)} W_{ij}^{(1)} + b_j^{(1)}$$

Activation ReLU

$$\hat{H}_j^{(s)} = \max(0, H_j^{(s)})$$

1.2 2nd Layer

Logits

$$Z_k^{(s)} = \hat{H}_j^{(s)} W_{jk}^{(2)} + b_k^{(2)}$$

1.3 Fused Softmax & Cross Entropy (*numerically stable*)

Inference

$$\hat{y}^{(s)} = \operatorname{argmax}_k Z_k^{(s)}$$

Note 1.2: At inference only $\hat{y}$ has to be computed.

Note 1.3: At train time this is purely cosmetic i.e. for logs or hyper-parameter training.

Numerical stability

$$a^{(s)} = \max_k Z_k^{(s)}$$

Predictions

$$p_k^{(s)} = \frac{e^{Z_k^{(s)} - a}}{\sum_{l=1}^C e^{Z_l^{(s)} - a}}$$

Note 1.4: p_k is a percentage ($p_k \in [0, 1]$) of certainty or believe of a certain prediction.

Fused gradients

$$G_k^{(s)} = \frac{\partial L}{\partial Z_k^{(s)}} = \frac{1}{N_{\text{local}}} (p_k^{(s)} - y_k^{(s)})$$

Note 1.5: The normalization $\left(\frac{1}{N_{\text{local}}}\right)$ is done here to control the gradient scale.

Cross Entropy Loss

$$L = - \sum_{k=1}^C y_k^{(s)} \log p_k^{(s)}$$

which since y_k is one-hot encoded can be written as:

$$L = - y_{k^*}^{(s)} \log p_{k^*}^{(s)} = - \log p_{k^*}^{(s)}$$

since $y_{k^*} = 1$ and with $p_{k^*}^{(s)} = p^{(s)}$.

Since we use the **numerically shifted Softmax**:

$$L = - \left(Z_{k^*}^{(s)} - a^{(s)} - \log \left(\sum_{l=1}^C e^{Z_l^{(s)} - a^{(s)}} \right) \right)$$

2 Backward Pass

Starting from:

$$G_k^{(s)} = \frac{\partial L}{\partial Z_k^{(s)}} = \frac{1}{N_{\text{local}}} (p_k^{(s)} - y_k^{(s)})$$

2.1 2nd Layer

Weight matrix gradient $\frac{\partial L}{\partial W_{jk}^{(2)}}$

$$\frac{\partial L}{\partial W_{jk}^{(2)}} = \frac{\partial L}{\partial Z_k} \frac{\partial Z_k}{\partial W_{jk}^{(2)}} = \frac{1}{R} \sum_{r=1}^R \sum_{s=1}^{N_{\text{local}}} \hat{H}_j^{(s)} G_k^{(s)}$$

Note 2.1: $\frac{1}{N_{\text{local}}}$ has already been applied in the definition of $G_k^{(s)}$.

Bias gradient $\frac{\partial L}{\partial b_k^{(2)}}$

$$\frac{\partial L}{\partial b_k^{(2)}} = \frac{\partial L}{\partial Z_k} \frac{\partial Z_k}{\partial b_k^{(2)}} = \frac{1}{R} \sum_{r=1}^R \sum_{s=1}^{N_{\text{local}}} G_k^{(s)}$$

Gradient flow $D_j^{H,(s)} = \frac{\partial L}{\partial \hat{H}_j^{(s)}}$

$$D_j^{H,(s)} = \frac{\partial L}{\partial \hat{H}_j^{(s)}} = \frac{\partial L}{\partial Z_k} \frac{\partial Z_k}{\partial \hat{H}_j^{(s)}} = W_{jk}^{(2)} G_k^{(s)}$$

Note 2.2: In order to adhere to DDP overlap, as soon as the weight matrix gradient or the bias gradient is computed locally an asynchronous `MPI_Allreduce` with `MPI_SUM` is started. This allows the send operation to overlap with the computation of the local 1st layer gradients before updating the global 2nd layer gradients.

2.2 1st Layer

Activation ReLU $\frac{\partial L}{\partial H_j^{(s)}} = \frac{\partial L}{\partial \hat{H}_j^{(s)}} \frac{\partial \hat{H}_j^{(s)}}{\partial H_j^{(s)}}$

$$D_j^{(s)} = \frac{\partial L}{\partial H_j^{(s)}} = \frac{\partial L}{\partial \hat{H}_j^{(s)}} \frac{\partial \hat{H}_j^{(s)}}{\partial H_j^{(s)}} = D_j^{H,(s)} \frac{\partial \hat{H}_j^{(s)}}{\partial H_j^{(s)}} = D_j^{H,(s)} \cdot \mathbb{I}[H_j^{(s)} > 0]$$

Weight matrix gradient $\frac{\partial L}{\partial W_{ij}^{(1)}}$

$$\frac{\partial L}{\partial W_{ij}^{(1)}} = \frac{\partial L}{\partial H_j^{(s)}} \frac{\partial H_j^{(s)}}{\partial W_{ij}^{(1)}} = \frac{1}{R} \sum_{r=1}^R \sum_{s=1}^{N_{\text{local}}} X_i^{(s)} D_j^{(s)}$$

Bias gradient $\frac{\partial L}{\partial b_j^{(1)}}$

$$\frac{\partial L}{\partial b_j^{(1)}} = \frac{\partial L}{\partial H_j^{(s)}} \frac{\partial H_j^{(s)}}{\partial b_j^{(1)}} = \frac{1}{R} \sum_{r=1}^R \sum_{s=1}^{N_{\text{local}}} D_j^{(s)}$$

3 Kernel / Implementation

The model is implemented through three distinct kernel which adhere to C++ template programming allowing to write one kernel for the forward pass through linear layers, one fused Softmax + CE and one backward pass through the linear layer.

3.1 Forward pass Linear Layer

Input: One or more samples i.e. vector or matrix respectively (X_i or $\hat{H}_j$).

Optional ReLU: Based on a template parameter `<bool FuseReLU>` ReLU is applied before the output in order to not write to device memory and reloading unnecessarily.

Output: A output vector ($\hat{H}_j$ hidden layer or Z_k).

This kernel is launched twice. Once for the first linear layer with `FuseReLU = true` and once for the second linear layer with `FuseReLU = false`.

3.2 Fused Softmax + CE

Input: Logits (vector: Z_k).

Output: Derivative $\frac{\partial L}{\partial Z_k}$ which is G_k .

Additional: The loss and correctness of the prediction as a byproduct for logging and hyper-parameter tuning. ($L^{(s)}$ loss per sample, and $C^{(s)}$ correctness per sample).

Inference Kernel:

At inference only the following is executed:

$$\hat{y}^{(s)} = \operatorname{argmax}_k Z_k^{(s)}$$

3.3 Backward pass Linear Layer

Input: The upstream gradient (G_k loss of Softmax or D_j^H).

Output: Gradients w.r.t. Weight matrix, and bias ($\frac{\partial L}{\partial W}$, and $\frac{\partial L}{\partial b}$)

Optional Gradient w.r.t. input: This only happens in the second layer since the gradient w.r.t. the model input is irrelevant ($\frac{\partial L}{\partial \hat{H}_j}$). This is indicated by another template parameter `<bool InputGrad>`.

Optional ReLU gradient: In case of the first layer the ReLU gradient is also fused into the kernel through a template parameter `<bool FuseReLU>` such that $\frac{\partial L}{\partial \hat{H}_j}$ is computed before computing the other gradients($\frac{\partial L}{\partial H_j}$ instead of $\frac{\partial L}{\partial \hat{H}_j}$).

This kernel is launched twice. Once for the second layer with `InputGrad = true` and `FuseReLU = false` and once for the first layer with `InputGrad = false` and `FuseReLU = true`.